

Three application families.

Make the task concrete.

Connect vision, recommendations, and time series, from movie examples to useful evaluation.

🕒 15-20 min core reading · 20-30 min practice · 12 short sections

Imagine running a small cinema app. It identifies posters from a camera, suggests movies to a viewer, and estimates tomorrow's ticket demand. These features share machine-learning ideas, but their inputs, outputs, mistakes, and evaluation rules differ. This chapter turns the earlier foundations into three concrete application designs.

WHAT YOU WILL UNDERSTAND

The main question: how do you choose the right prediction task and prove that its results are useful?

The practical path: match boxes to objects, score and rank eligible movies, and forecast demand using only information available at the time.

Reading path: twelve short sections, three diagrams, three interactive experiments, and a Python workbook. Optional notes add depth without interrupting the core explanation.

1. Start with the decision the application must support

A useful design begins with an **input, output, decision, and cost of error**. “Use AI for our cinema” does not yet specify a learning problem.

Feature	Input and output	Decision supported	A meaningful mistake
Poster finder	Image → poster locations	Which region should be cropped for identification?	Missing a small poster or returning two boxes for one poster.
Movie recommender	Viewer history + eligible catalog → ranked list	What should appear in the first few positions?	Ranking a poor fit first, or offering an unavailable movie.
Demand forecast	Earlier daily ticket counts → future counts	How much capacity should tomorrow’s schedule provide?	Underestimating a busy day or staffing for demand that never arrives.

The **unit of evaluation** changes: objects in images, ranked items per viewer, or forecasts at specific times and horizons. One generic “accuracy” score cannot describe all three. Keep a simple baseline and an explicit evaluation contract for each, building on Chapter 34.

Our lab uses authored boxes, movie scores, and 21 daily counts. It calculates real matching, ranking, and forecasting metrics. It does not train or run a vision network or a collaborative-filtering model.

2. Turn pixels into the output the task needs

An image is a grid of numerical values, usually with color channels. A vision model learns features that connect those values to a target. Chapter 22 introduced CNNs and transfer learning; the application still needs the right kind of target.

Vision task	Question	Cinema example
Classification	What label describes this image or crop?	Classify a cropped poster into a catalog category.
Object detection	What objects occur, and where?	Return a box and class for each poster on a wall.
Semantic segmentation	What class does each pixel belong to?	Label every poster pixel, without separating individual posters.
Instance segmentation	Which pixels belong to each individual object?	Separate two overlapping posters with two masks.
OCR / image retrieval	What text is visible / which catalog image is similar?	Read a title, or find a candidate catalog match from a crop.

➤ **One image can support different prediction tasks.**

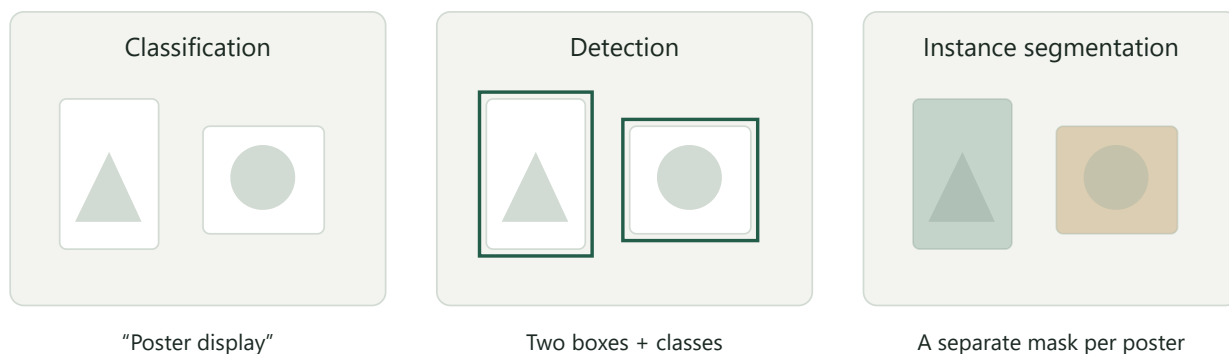


Figure 1. Classification assigns a label to an image; detection encloses each object; instance segmentation identifies each object's pixels. These drawings illustrate target types, not a model's output.

A **bounding box** encloses an object; a **mask** labels its pixels. Detection can locate a poster without knowing its movie title. A later OCR or image-embedding retrieval step can propose catalog identities, followed by validation. Similar artwork or unreadable text can still produce ambiguity. See the [TorchVision detection and segmentation tutorial](#) for a concrete image/box/mask training workflow.

▼ **Go deeper: data, transfer learning, and image transformations**

Start with labeled examples that represent actual use: camera angles, small objects, glare, blur, and lighting. Use the same box and class definitions across annotators. A pretrained backbone can supply visual features; a task head learns the required outputs. Freezing a backbone trains only the head, while fine-tuning updates selected pretrained weights too.

Resize, crop, or flip an image together with its boxes and masks. A crop can remove an object, and a flip changes coordinates. Keep related frames from one recording together when splitting data; almost identical frames on both sides can inflate evaluation. Also inspect separate cameras or locations when deployment must generalize to them. These are application-specific versions of the leakage and transfer-learning lessons from Chapters 12 and 22.

3. Measure whether the detected object is in the right place

The lab stores boxes as **[left, top, right, bottom]** coordinates. **Intersection over union (IoU)** measures overlap:

$$\text{IoU}(A, B) = \frac{\text{area}(A \cap B)}{\text{area}(A \cup B)}.$$

Identical boxes have IoU 1; nonoverlapping boxes have 0. For two 10-by-10 boxes shifted horizontally by 2, the intersection is 80 and the union is 120, so IoU is $80/120 \approx 0.667$. This measures location agreement, not how confident the detector feels. See the [TorchVision IoU definition](#).

Our simplified **single-class evaluation** keeps predictions whose score meets a threshold, processes them from highest score downward, and matches each to the best still-unmatched truth box in the same frame with IoU at least 0.5. Each truth object can be matched once. A matched prediction is a true positive (TP); an unmatched prediction is a false positive (FP); an unmatched truth object is a false negative (FN).

At score threshold **0.6**, five predictions survive: **3 TP, 2 FP, and 1 FN**. Precision is $3/5 = 60\%$; recall is $3/4 = 75\%$. P2 overlaps an already matched object, so it is a duplicate FP. P7 has a high score but matches no object.

The **score threshold** decides which predictions to keep; the **IoU threshold** decides whether their locations count as correct. Raising the score threshold to 0.9 leaves one TP and one FP here: precision falls to 50% and recall to 25%. A higher threshold does not guarantee higher precision on a finite dataset, and a score is not automatically a calibrated probability.

Real detector evaluations also use average precision (AP), summarizing precision/recall across score cutoffs; **Mean average precision (mAP)** averages AP across classes and, in some protocols, IoU thresholds. Always state the protocol. [Non-maximum suppression](#) can remove overlapping duplicate predictions before evaluation; it is different from matching predictions to ground truth. This lab intentionally retains a duplicate so the rule is visible.

4. Build recommendations from candidates to a ranked list

Content-based recommendations compare item properties with a viewer's interests. A viewer who liked gentle adventures may like another movie with similar attributes. **Collaborative filtering** learns from interaction patterns across users and items; it can discover a movie that looks different in metadata but appeals to similar viewers. **Hybrid systems** combine signals. See Google's introductions to [candidate generation](#) and [collaborative filtering](#).

➤ Retrieve candidates. Score them. Build the list.

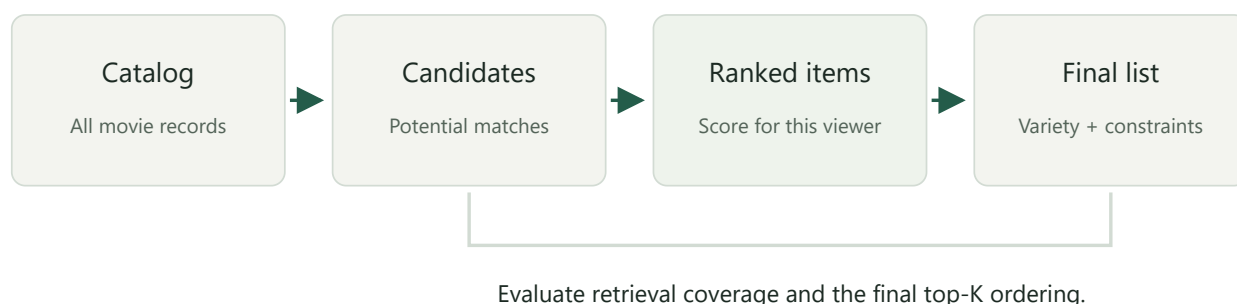


Figure 2. A practical recommender narrows the catalog, ranks candidates, and applies list-level rules. Eligibility can be enforced at several stages. The lab starts with the full eligible pool and evaluates its final ordering.

Candidate generation retrieves a manageable pool. A ranker scores the candidates using richer context. Re-ranking applies list-level goals such as variety and freshness, while hard eligibility rules enforce availability and other requirements. A ranker cannot recover a relevant item that candidate generation omitted. This follows the candidate, scoring, and re-ranking architecture.

In our lab, **all five eligible movies are candidates**. Already watched and unavailable items are excluded before scoring. A hand-authored interest vector is compared with movie feature vectors by cosine similarity. The other input is an authored collaborative-style score, not a score fitted from user logs. Both are on a 0-1 teaching scale:

$$s(u, i) = \lambda s_{\text{content}}(u, i) + (1 - \lambda) s_{\text{collaborative}}(u, i).$$

For *Starlit Journey*, content is 0.6 and the collaborative-style score is 0.9. At $\lambda = 0.5$, the combined score is **0.75**. Higher ranks win; exact ties use movie ID. In production, signals need comparable scales or a learned ranker; averaging arbitrary raw scores is not a general recipe.

▼ Go deeper: latent factors, implicit feedback, and cold start

Matrix factorization represents each user and item with learned latent vectors; their dot product estimates affinity. The factors need not correspond to named genres. Explicit feedback might be a rating; implicit feedback might be a watch or click. Missing feedback does not prove dislike: the viewer may never have seen the item. See Google's [matrix-factorization explanation](#).

Cold start means too little history for a user or item. New-user onboarding, contextual or broadly useful defaults, and content features can provide initial candidates. A new movie can have a useful description before it has interactions. The lab's feature names are deliberately interpretable; learned embeddings usually need a different interpretation. No LLM is required to compute these ranking scores. An LLM may help explain a verified recommendation, but its explanation does not establish that the ranking is good.

5. Evaluate the list the viewer actually receives

Choose a relevance definition and a cutoff **K**. Here, two eligible movies are marked relevant in an authored later evaluation snapshot. Those labels enter the metrics only, never the ranking score.

$$\text{Precision@K} = \frac{\text{relevant items in top K}}{K}, \quad \text{Recall@K} = \frac{\text{relevant items in top K}}{\text{all relevant eligible items}}.$$

At the default blend, the top two are *Starlit Journey* (relevant) and *Quiet Orbit* (not relevant). Precision@2 and Recall@2 are both **0.5**. The ideal top two would contain both relevant movies.

Normalized discounted cumulative gain (NDCG@K) rewards putting relevant items earlier. For our binary labels, each relevant item at position r contributes $1/\log_2(r + 1)$. Divide this discounted gain by the gain of the ideal ordering. A relevant item first and an irrelevant item second gives $1/(1 + 1/\log_2 3) \approx \mathbf{0.613147}$. Reversing them gives **0.386853**, even though precision and recall are unchanged. See the [NDCG metric documentation](#).

Increasing K to 3 with the default blend includes *Harbor Lights*: precision becomes 2/3 and recall 1. Different list sizes serve different user experiences. Compare systems at an agreed K and candidate universe; use multiple viewers, and state how users with no relevant items are handled. Our evaluator returns undefined for recall/NDCG if no eligible relevant items exist.

▼ Go deeper: exposure bias and feedback loops

An unclicked movie may be irrelevant, unseen, or simply ranked too low to notice. Training directly on clicks can reinforce earlier exposure and popularity. A recommender changes what users see, which changes the next training dataset. Google's [scoring guide](#) discusses objectives and position bias.

Use time-aware evaluation with historical features and later outcomes, inspect new users/items and other important groups, and monitor diversity, catalog coverage, unwanted repetition, and satisfaction. Carefully designed online experiments measure effects that offline ranking scores alone cannot establish. Our one-viewer example illustrates arithmetic, not recommendation quality for a population. Chapter 38's warning about optimizing a proxy applies here too.

6. Represent demand as a time series

A **time series** is a sequence of observations indexed by time. Specify what is counted, the sampling interval, and when each value becomes available. "Daily ticket demand" could mean bookings made that day, tickets for that day's shows, or actual attendance; these are different targets.

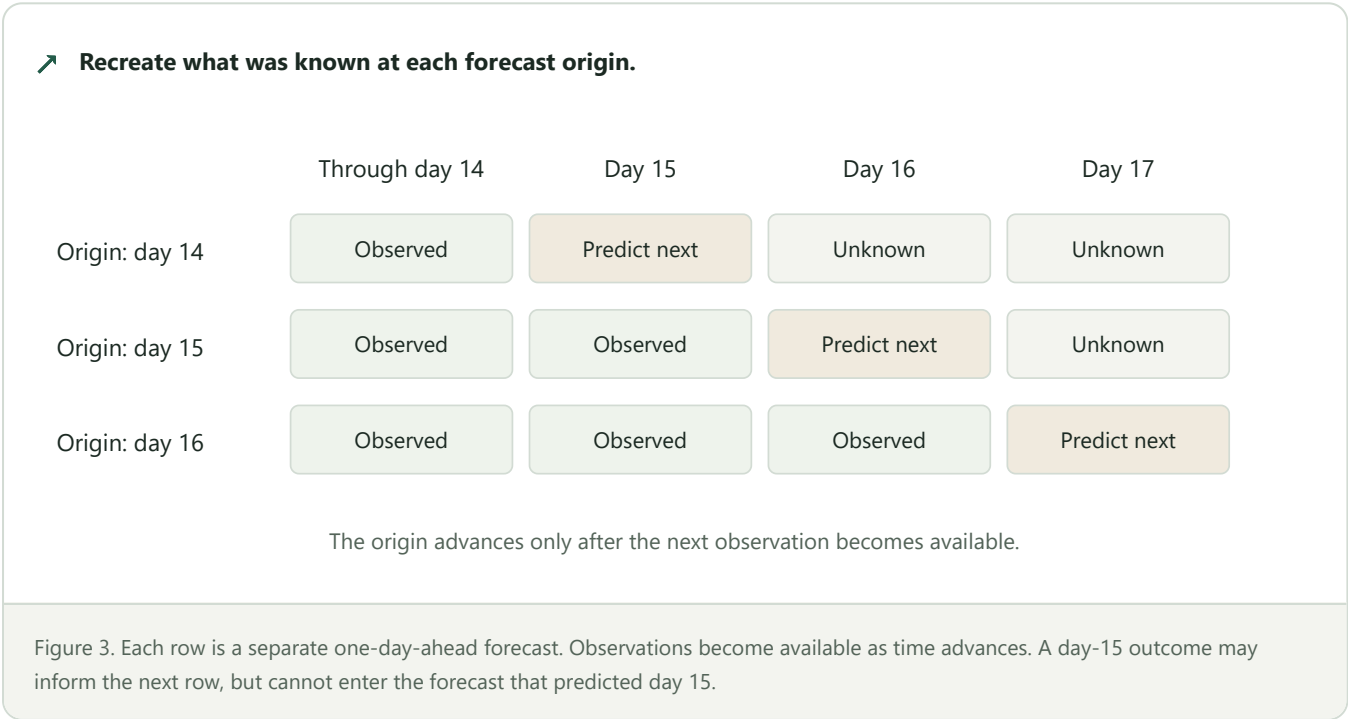
Trend is longer-term movement. **Seasonality** repeats at a known period, such as a weekly pattern. The **remainder** is what the chosen decomposition leaves unexplained; it is not proof of meaningless noise. See [time-series components](#). A 21-day example can demonstrate a weekly pattern, but cannot establish yearly seasonality.

For a forecast made after day t , a lag-one feature is y_t , and a trailing-three-day mean is $(y_t + y_{t-1} + y_{t-2})/3$. Forecasting tomorrow must not use tomorrow's count. A centered moving average that includes future observations would leak information.

Calendar features such as day of week may be known in advance. Actual future weather or final bookings are not automatically known at forecast time; use only the version available then. Distinguish a missing report from zero tickets and from a closed cinema. Consistent timestamps, time zones, and reporting cutoffs are part of the modeling problem.

7. Forecast with baselines and honest backtests

Start with forecasts you can explain. A **last-value baseline** predicts tomorrow equals today. A **seasonal-naive baseline** predicts tomorrow equals the same weekday last week. Our **trailing mean of three** averages the last three observed days. Simple baselines are essential comparisons for more complex methods; see [simple forecasting methods](#).



The **forecast origin** is when the prediction is made; the **horizon** is how far ahead it predicts. Our lab evaluates seven **one-day-ahead** forecasts, for days 15-21. To predict day 15, it uses observations through day 14. After day 15 is observed, it may use that value to predict day 16. This is a **rolling-origin backtest**, not seven forecasts all issued on day 14.

For day 15, last value predicts **105**; trailing mean predicts **108.333333**; seasonal naive uses day 8 and predicts **45**. The actual count is **50**. The weekly baseline handles this deliberately weekly dataset well; that is not evidence that it wins on every cinema.

Freeze each method before scoring a fold. Any fitted scaler, imputer, feature selector, or model must use only information available at that origin. Tuning repeatedly on these seven displayed outcomes turns them into development data; keep a later untouched period for a final assessment. Randomly shuffling rows is generally unsuitable for estimating future forecasting performance when it allows later information into earlier predictions.

▼ Go deeper: statistical, feature-based, and neural forecasters

Exponential smoothing gives more weight to recent observations and can model level, trend, and seasonality. ARIMA (autoregressive integrated moving average) models dependence on earlier values and past errors, with differencing used to address certain nonstationary patterns; seasonal extensions handle periodic structure. Roughly, stationarity means statistical properties remain stable over time. ETS (error, trend, seasonal) models and ARIMA offer different representations; see the [ETS/ARIMA comparison](#).

A regression or tree model can use lagged counts, calendar variables, and other available predictors. Neural sequence models can share patterns across many related series. Data volume, noise, changing conditions, and the forecast horizon determine whether added complexity helps. Compare all candidates under the same information cutoff. For a seven-day forecast issued today, later actual counts remain unknown: use an appropriate multi-step method and evaluate each horizon. Do not silently substitute observed future counts between those steps.

8. Connect forecast errors to decisions and uncertainty

For actual y_t and forecast $\hat{y}_t$, define error $e_t = y_t - \hat{y}_t$. Positive error means underprediction here. **MAE** averages absolute errors; **RMSE** takes the square root of mean squared errors and gives large misses more influence. Both have the target's units. See [forecast accuracy measures](#).

Lab method	MAE over days 15-21	RMSE	What the result means here
Last value	22.143 tickets	27.710	Yesterday can be a poor guide across weekday changes.
Trailing mean of three	31.905 tickets	37.054	Smoothing the recent weekend can overpredict the next quiet day.
Seasonal naive, lag seven	5.000 tickets	5.000	Every forecast is five below the actual in this authored series.

MAE 5 does not mean every possible future error is at most 5. MAPE divides by actual values and becomes problematic at or near zero; it is not a universal replacement. Evaluate the required horizons, peaks, quiet periods, and different cinemas, and keep measurement units visible.

A forecast estimates demand; a scheduling policy decides capacity using costs and constraints. Likewise, a detected poster still needs identity validation, and a recommendation must satisfy eligibility rules. For each application, preserve the input version, prediction, decision, later outcome, and time needed to detect performance changes.

▼ Go deeper: prediction intervals and changes in demand

A **prediction interval** describes uncertainty about a future observation under a model and its assumptions. It is different from a confidence interval for an estimated parameter. A nominal 90% interval should be assessed for roughly 90% coverage on comparable future cases, alongside its width; an extremely wide interval may cover well but be unhelpful. See [prediction intervals](#).

New releases, a changed timetable, holidays, or a closure can alter demand. Track recent errors and missing inputs; investigate shifts before simply retraining. Underprediction and overprediction may have different operational costs, so a chosen quantile can be more useful than a single central estimate. Our seven toy errors cannot support a reliable general uncertainty claim; the lab intentionally reports point forecasts without invented intervals.

9. Run the Python applications workbook

Extract the ZIP and open a terminal in its folder. Use Python 3.10 or later; no packages, images, credentials, or model calls are needed.

```
python applications.py
python applications.py --threshold 0.9
python applications.py --weight 0 --k 2
python applications.py --weight 0.5 --k 3
python applications.py --forecast last
python -m unittest -v test_applications.py
```

File	Purpose
<code>data.json</code>	Truth boxes, supplied detections, fictional movie signals, and daily counts.
<code>applications.py</code>	IoU matching, hybrid ranking, top-K metrics, and rolling forecasts.
<code>test_applications.py</code>	Hand calculations, duplicate handling, ranking isolation, and time-cutoff checks.
<code>README.md</code>	Commands, expected results, and the example's assumptions.

The report separates `vision`, `recommendations`, and `forecast`. Inspect prediction matches, ranked scores, and each forecast's `sourceDays`. Scores use full precision for calculation and ordering; exported numbers round to six decimals. Changing a control changes only its corresponding task, and each run starts fresh.

10. Experiment with the cinema applications lab

Same cinema. Three different tasks.

Default settings: detector threshold 0.6; content weight 0.5; $K = 2$; seasonal-naive forecast with lag seven.

Experiment	Default result	Change to inspect
Vision	3 TP, 2 FP, 1 FN. Precision 60%; recall 75%.	At threshold 0.9: 1 TP, 1 FP, 3 FN.
Recommendations	Precision/recall 0.5; NDCG 0.613147.	At $K = 3$: precision 2/3; recall 1; NDCG 0.919721.
Forecasting	MAE = RMSE = 5 tickets.	Last value: MAE 22.143; mean three: 31.905.

Day 15's seasonal forecast uses day 8: predicted 45, actual 50. This is rolling one-day evaluation, not a seven-day forecast issued once.

Exercise 1: distinguish confidence from correctness

In Vision, compare thresholds 0.6 and 0.9. Why does the higher threshold fail to improve precision? Find the duplicate prediction at the default setting.

▼ Reveal the explanation

At 0.6 there are 3 TP, 2 FP, and 1 FN. At 0.9, only P1 and P7 remain: 1 TP, 1 FP, and 3 FN. P7 is confidently wrong. P2 is a duplicate because P1 already matched A1. Score and spatial correctness measure different things.

Exercise 2: compare equal hit counts with different orderings

In Recommendations, keep $K = 2$ and change the content weight from 0 to 0.5. Then increase K to 3 with weight 0.5. Explain the metric changes.

▼ Reveal the explanation

At $K = 2$ both weights find one of two relevant movies, so precision and recall stay 0.5. Moving Starlit Journey from second to first raises NDCG from 0.386853 to 0.613147. At weight 0.5 and $K = 3$, Harbor Lights enters: precision is 2/3, recall is 1, and NDCG is 0.919721.

Exercise 3: audit one forecast before trusting the average

In Forecasting, inspect day 15 for each method. Which observations does each use? Would using the real day-15 value to predict day 16 always be allowed?

▼ Reveal the explanation

Last value uses day 14; mean three uses days 12-14; seasonal naive uses day 8. All precede day 15. The day-15 actual may be used once it is observed in this rolling one-day setup. It would be future leakage for a day-16 forecast already issued on day 14. The origin determines what is allowed.

11. Check your understanding

01 A second prediction overlaps a poster already matched by a higher-scoring prediction. How is it counted in this lab?

- A. A duplicate false positive.
- B. A second true positive for the same poster.
- C. A false negative.

Answer: A. Each truth object can be matched once. Repeated predictions do not create additional correctly found objects.

02 Two top-two lists each contain one relevant movie. Why can their NDCG differ?

- A. NDCG ignores relevance.
- B. NDCG gives earlier relevant items more weight.
- C. NDCG uses ticket demand.

Answer: B. At $K = 2$, a relevant item first gives 0.613147 here; the same item second gives 0.386853. Precision and recall are unchanged.

03 What should happen to an unavailable movie with the highest recommendation score?

- A. Always show it first.
- B. Treat its score as proof of availability.
- C. Exclude it using the eligibility rule.

Answer: C. A learned or authored score estimates preference. It does not override the application's hard eligibility requirements.

04 When can an observed day-15 count be used to forecast day 16?

- A. For any forecast labeled day 16, regardless of issue time.
- B. When the forecast is issued after that count becomes available.
- C. Only when the forecast uses a neural network.

Answer: B. Information availability at the forecast origin governs the decision. A forecast issued on day 14 cannot use the later actual count.

05 The seasonal baseline has MAE 5 on these seven forecasts. What can you conclude?

- A. Every future error is at most five tickets.
- B. It is the best forecasting method for every cinema.
- C. Its mean absolute error is five tickets on this evaluation window.

Answer: C. The result describes this authored window and one-day horizon. It is neither an error bound nor a general claim about future performance.

12. Revise and connect the ideas

Explain the three contracts: **vision matches predicted objects to labeled objects; recommendations evaluate an ordered eligible list; forecasting evaluates future values using information available at each origin.**

Then explain why an attractive model score does not by itself establish a useful decision.

▼ Quick-reference glossary

Term	Meaning
Box / mask	Object-enclosing rectangle / pixel-level region labels.
IoU	Intersection area divided by union area.
Score / IoU threshold	Prediction-selection cutoff / spatial-match cutoff.
AP / mAP	Precision-recall summary / average across a stated evaluation protocol.
Content / collaborative filtering	Item-property signals / user-item interaction signals.
Candidate generation / ranking	Retrieve a pool / order the pool.
Cold start	Too little user or item interaction history.
Precision@K / Recall@K	Relevant fraction of the list / fraction of relevant items retrieved.
NDCG@K	Discounted ranking gain divided by ideal gain.
Lag / seasonality	Earlier observation / repeating pattern at a known period.
Origin / horizon	Prediction issue time / distance into the future.
Backtest	Evaluation that recreates earlier forecast decisions and information cutoffs.
MAE / RMSE	Absolute-error average / square root of squared-error average.
Prediction interval	A range expressing uncertainty about a future observation.

The linked TorchVision, Google, scikit-learn, and forecasting references provide deeper implementation paths. The cinema data, diagrams, calculations, and workbook are original teaching examples. Revisit Chapters 15, 17, 22, and 34 for the evaluation, representation, architecture, and system-level foundations.

Next: Chapter 40 - Graphs, causality, and alternative learning methods. We will examine relationships, interventions, and learning setups beyond the standard supervised pipeline.